

VISTRAN – Need-Based Teacher Training Platform

Hackathon Submission – Theme 2: DIET & SCERT Teacher Training

1. Executive Summary

The Challenge: Dr. Kumar, a DIET principal, struggles with "training fatigue." Teachers find centralized, 50-page training manuals irrelevant to their specific classroom realities (Tribal vs. Urban). He lacks a real-time feedback loop to know what problems teachers are actually facing.

The Solution: VISTRAN is a dynamic, AI-powered platform that replaces static manuals with a "Live Training Ecosystem." It allows teachers to report specific issues, uses AI to adapt training content to the local context (Language/Culture), and provides instant AI support—all within a unified Java-based architecture.

2. Our Approach

VISTRAN shifts the paradigm from "Supply-Driven" to "Demand-Driven" training.

A. Problem-First Methodology

Instead of pushing random training modules, VISTRAN pulls data from the classroom first.

- **Step 1:** Teachers report immediate blockers (e.g., "Student Absenteeism," "Hard Science Concepts") via the **Issue Reporting System**.
- **Step 2:** The system aggregates this data for Administrators (DIET Principals) to visualize cluster-wise trends.
- **Step 3:** Training is assigned *in response* to these specific data points.

B. Demographic Contextualization

We recognize that "Cluster A (Urban)" and "Cluster C (Tribal)" have different needs.

- **Contextual AI:** We use Generative AI not just to summarize, but to *rewrite* content. A standard science lesson is automatically simplified and culturally adapted for a tribal context using familiar local examples, ensuring higher engagement.

C. Role-Based Architecture

The platform provides distinct, secure workflows for every stakeholder:

- **Admin:** High-level analytics and training deployment (AdminDashboardServlet).
- **Trainer:** Content creation and issue resolution (TrainerSubjectDashboardServlet).

- **Teacher:** Learning access, issue raising, and AI assistance (RaiseIssueServlet).
-

3. Live Deployment

Please click the link below to view the live prototype of VISTRAN.

 **Deployment Link:** vistran.koyeb.app/

 **GitHub Repository:** <https://github.com/phantom550/Vistran>

4. Solution in Detail (Technical Implementation)

A. Smart AI Content Adaptation (Java Service)

File: `src/main/java/com/user/PollinationsService.java`

This dedicated Java service intercepts standard training materials and processes them through the Pollinations.ai API to generate three distinct versions:

- **Urban (Cluster A):** Modern, practical language with high-resource examples.
- **Semi-Urban (Cluster B):** Balanced language with everyday Indian examples.
- **Tribal (Cluster C):** Simplified vocabulary, short sentences, and culturally familiar analogies.

B. Integrated AI Chat System (Servlet)

File: `src/main/java/com/user/GeminiChatServlet.java`

- **Replacement:** This Servlet completely replaces the previous Node.js server.
- **Function:** Handles all chatbot interactions directly within the Tomcat container. It acts as an intelligent "Pedagogical Assistant," helping teachers troubleshoot classroom management issues instantly without waiting for a human trainer.

C. Real-Time Feedback Loop

File: `src/main/java/com/user/RaiseIssueServlet.java`

- **Function:** Captures specific classroom issues tagged by Class and Subject.
 - **Impact:** Reduces the feedback cycle from months (paper reports) to seconds (database entry), allowing DIETs to react immediately.
-

5. System Architecture

The system follows a pure **Java MVC (Model-View-Controller)** architecture:

- **View (Frontend):** JSP Pages (clusterC.jsp, subjectdashboard.jsp), HTML5, CSS3, Material Design.
 - **Controller (Backend):** Jakarta EE Servlets manage data flow and AI logic.
 - GeminiChatServlet: AI Chatbot controller.
 - AdaptModuleServlet: Training content adaptation controller.
 - **Model (Data):** MySQL Database accessed via DBConnection.java (Singleton Pattern).
-

6. How to Run Locally

Follow these steps to run the application on your local machine.

Prerequisites

- **Java Development Kit (JDK):** Version 21 or higher.
- **Web Server:** Apache Tomcat 10.1.
- **Database:** MySQL Server 8.0.
- **IDE:** Eclipse IDE (Enterprise Java edition).

Step 1: Database Configuration

1. Open MySQL Workbench.
2. Create the database:

SQL

```
CREATE DATABASE void4;
```

3. Import the SQL schema provided in the /database folder of the repo.
4. **Note:** The app connects via jdbc:mysql://localhost:3306/void4 (User: root, Pass: root) by default. Update src/main/java/com/user/DBConnection.java if your credentials differ.

Step 2: Server Setup (No Node.js Required)

1. Open **Eclipse IDE**.
2. Import the project: File > Open Projects from File System > Select 'vistran by void4'.
3. Right-click the project > Build Path > Configure Build Path.
 - Add Apache Tomcat v10.1 to Libraries.

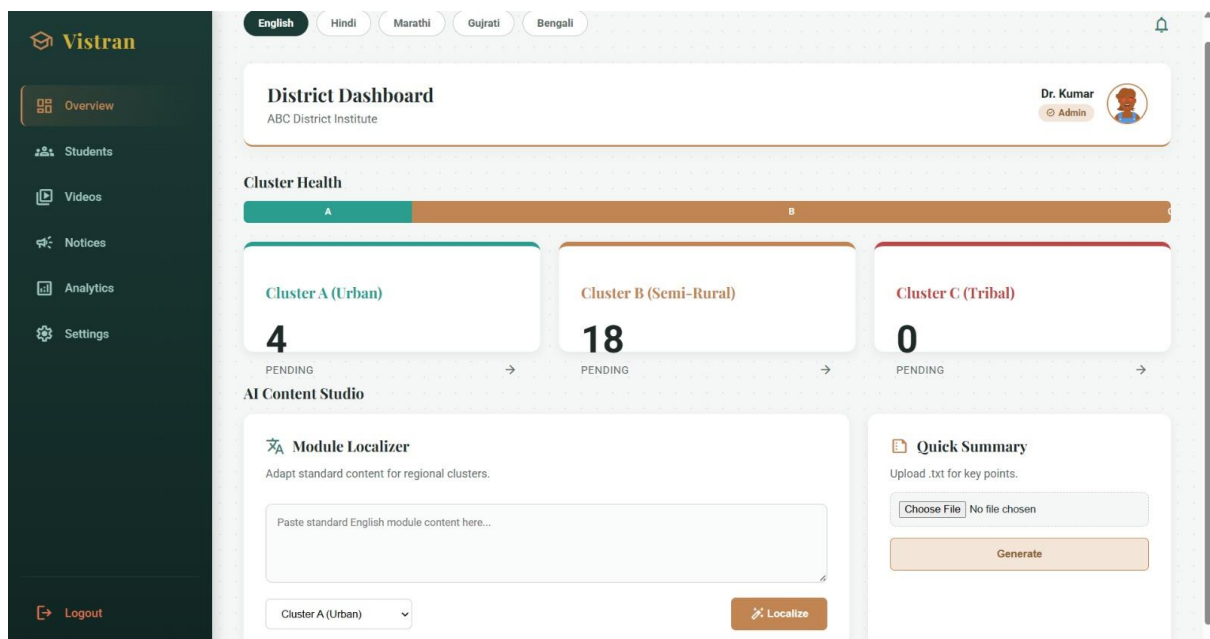
- Add mysql-connector-j-9.5.0.jar (from WEB-INF/lib) to Classpath.
4. Right-click Project > Run As > Run on Server.
 5. Select **Tomcat v10.1**.

Step 3: Access the Platform

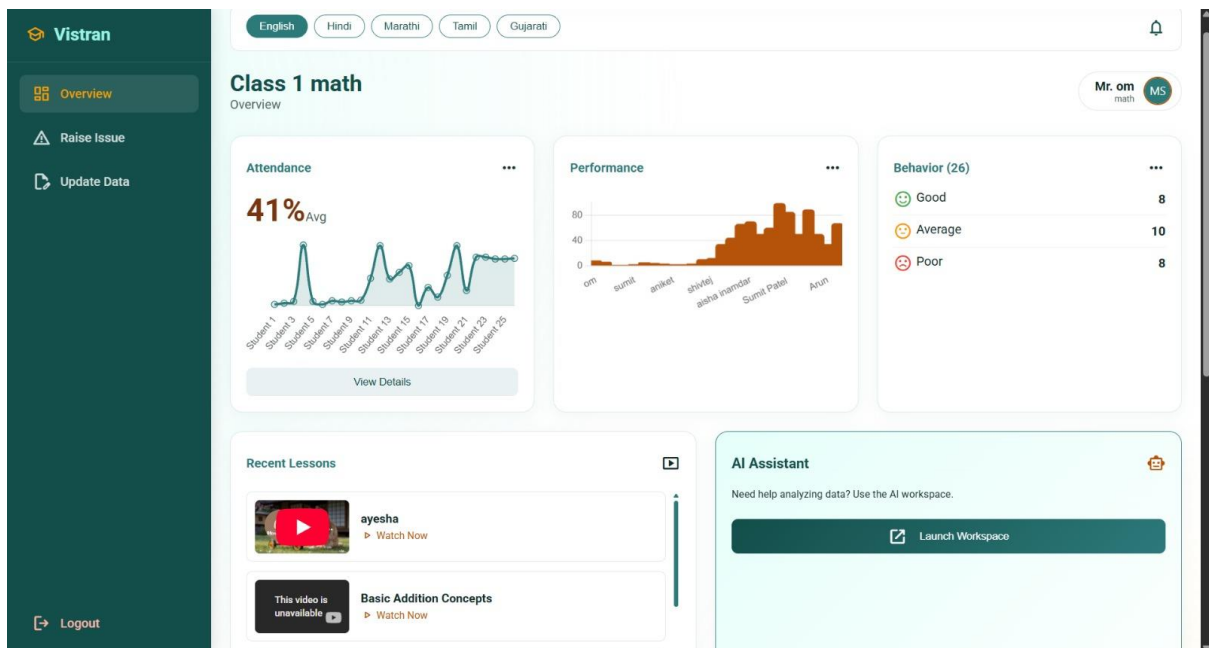
- **Home URL:** http://localhost:8080/vistran_by_void4/
- **Admin Login:** Access via adminlogin.jsp.

7. Project Screenshots

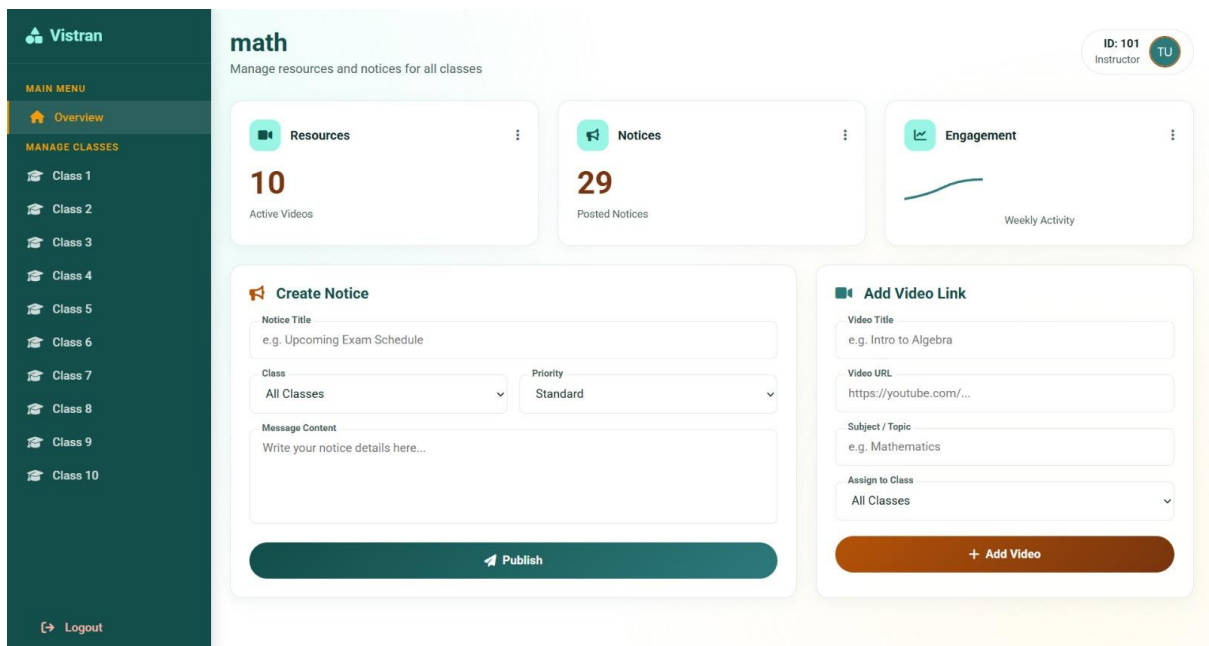
A. Admin Dashboard Overview



C. Teacher Dashboard Overview



C. Trainer Dashboard Overview



8. Tech Stack

- **Frontend:** JSP, HTML, CSS, JavaScript
- **Backend:** Java (Jakarta Servlets)
- **Database:** MySQL
- **AI Service:** Pollinations.ai (via Java HTTP Client)
- **Server:** Apache Tomcat 10.1

Team Members

1. Om Chaudhari – GitHub: <https://github.com/om0709>
2. Ayush Yadav – Github <https://github.com/phantom550>
3. Siddhi Bahutule – GitHub: <https://github.com/siddhivilas26>
4. Aisha Inamdar – GitHub: <https://github.com/BUGBUSTER11>